



AI Forensic Duplicate Detection CLI — Developer Documentation

1. Overview

The **AI Forensic Duplicate Detection CLI** (codenamed *Dupli-HQ*) is a modular command-line interface (CLI) for: - **File-level forensic analysis** - **Duplicate detection** - **Automated folder monitoring** - **Daily AI-enhanced snapshots**

It integrates **traditional hashing** (MD5/SHA1/SHA256) with **AI-based perceptual hashing and embeddings** using models like **CLIP**, **DINOv2**, **ResNet**, **EfficientNet**, **SBERT**, and **CodeBERT**.

2. Core Architecture

Directory Overview

```
src/
├── ai_model/
│   ├── resnet18_model.py
│   ├── resnet50_model.py
│   ├── efficientnet_b1_model.py
│   ├── clip_model.py
│   ├── sbert_deep_model.py
│   └── codebert_model.py
├── cli_tool/
│   ├── automation/
│   │   ├── daily_snapshot.py
│   │   ├── scan_duplicates.py
│   │   └── folder_tracker.py
│   ├── hashing/
│   │   ├── hash_utils.py
│   │   └── perceptual_hash.py
│   └── interface/
│       ├── cli_shell.py
│       ├── commands.py
│       └── logger.py
└── loader.py
```

Each part of the system is imported dynamically via `loader.py`, ensuring that paths stay valid even in non-standard directory structures.

3. CLI Entry Points

A. `cli_shell.py`

Interactive shell interface (`dupli-hq`) that supports live execution of snapshot, duplicate, and tracker modes.

Commands:

- `set path <folder>` — Set target directory
- `set mode <snapshot|duplicates|tracker>` — Define operation mode
- `set strategy <full|custom>` — Reserved for future batch configurations
- `show` — Display current settings
- `run` — Execute selected mode
- `exit` — Quit CLI

Behavior:

- Logs every command via `logger.py`
 - Runs diagnostics through `boot_diagnostics()` to ensure AI models are available
 - Uses color-coded output for clarity
 - Interfaces directly with:
 - `daily_snapshot.main()`
 - `scan_duplicates.scan_folder_for_duplicates()`
 - `tracker.monitor_folder()`
-

B. `commands.py`

Non-interactive command-line interface using `argparse` — suitable for automation or scripting.

Modes:

1. `--mode snapshot`
→ Calls `daily_snapshot.main(folder)`
2. `--mode duplicates`
→ Scans and saves duplicate report as JSON under `/reports/scan/`
3. `--mode tracker`
→ Continuously monitors folder for tampering and duplicates
4. `--mode compare`
→ Compares two files using AI embeddings

Options:

- `--auto` → Automatically select model based on file type (image/text/code)
- `--threshold` → Similarity threshold (default 0.9)

- `--model` → Force specific model
 - `--folder` / `--file1` / `--file2` → Input sources
-

4. Automation Modules

A. `daily_snapshot.py`

Creates an AI-augmented snapshot of all files in a folder.

Each file entry contains:

```
{
  "mode": "AIHASH",
  "hash": "<sha256>",
  "value": [<optional embedding vector>]
}
```

Key Features:

- AI-based embeddings for image and text/code files
 - Byte-level change detection via SHA256
 - Cosine similarity comparison for AI vectors
 - Generates differential reports (`/reports/diffs/`)
 - Can load previous snapshots automatically and detect changes
-

B. `scan_duplicates.py`

Scans a folder for identical or perceptually similar files using hybrid AI models.

Algorithm:

1. Group files by type (image, text, code)
2. For each pair:
3. Compare **SHA256** for exact duplicates
4. Use **pHash** + AI embeddings for perceptual duplicates
5. Compute **cosine similarity** across feature vectors

Models Used:

- **Image:** DINOv2, ResNet50
- **Text/Code:** SBERT-Deep + CodeBERT (feature concatenation)

Outputs:

- JSON report stored in `/reports/scan/duplicates_TIMESTAMP.json`
-

C. `folder_tracker.py`

Live monitoring daemon that: - Periodically snapshots target folders - Detects tampering, additions, deletions - Logs results in `/reports/tracker_alerts.txt` - Optionally re-runs duplicate scanning after change detection

Default check interval:

`CHECK_INTERVAL = 30 seconds`

5. Hashing Modules

A. `hash_utils.py`

Traditional cryptographic hash utilities: - `compute_md5`, `compute_sha1`, `compute_sha256` - `scan_directory(directory)` — returns all hashes per file

B. `perceptual_hash.py`

AI-independent visual hash for images.

Functions:

- `compute_phash(path)` → 64-bit perceptual hash
- `hamming_distance(h1, h2)`
- `analyze_perceptual_hash_similarity(h1, h2)` → returns category flag (identical/similar/different)

Used for **fast pre-screening** before deep model comparison.

6. Logging & Diagnostics

`logger.py`

Every user command (CLI or shell) is recorded under:

`src/logs/cli_commands.log`

Format:

```
[YYYY-MM-DD HH:MM:SS] <command>
```

Boot Diagnostics

When the CLI starts, it:

1. Loads all automation modules
2. Tests AI model imports
3. Displays initialization status (`OK` or `Failed`)

7. Dynamic Loader

```
loader.py
```

Finds project root via `.git`, `pyproject.toml`, or `requirements.txt`, then dynamically imports every required module.

Benefits:

- No hardcoded import paths
- Safe under varying environments
- Supports lazy loading (`__getattr__`) for performance

8. AI Models Integration

Supported Models and Expected Behavior

Model	Type	Purpose	Expected Behavior
ResNet18 / 50 / 101	CNN	Visual embeddings	Coarse-to-fine similarity for image files
EfficientNet B1 / B3	CNN	Visual embeddings	Lightweight high-accuracy backup
CLIP	Multimodal	Cross-modal embeddings	Detects perceptual similarity between images
DINOv2	Vision Transformer	Self-supervised image embedding	High-robustness for image duplicates
SBERT / SBERT-Deep	Transformer	Text embedding	Captures semantic similarity between documents

Model	Type	Purpose	Expected Behavior
CodeBERT	Transformer	Code embedding	Detects structural and functional similarity in source code

Each model exposes: - `load_model()` — returns initialized model (and tokenizer/preprocess if needed)
- `extract_features()` / `extract_features_from_file()` — produces normalized embedding vectors

9. Reports and Output Files

All generated artifacts are stored under:

```
/reports/
├─ snapshots/      → per-folder AIHASH snapshots
├─ diffs/          → snapshot difference reports
├─ scan/           → duplicate scan reports
└─ tracker_alerts.txt
```

10. Typical Workflows

A. Manual CLI Mode

```
python commands.py --mode duplicates --folder ./data
```

B. Interactive Shell Mode

```
python cli_shell.py
dupli-hq> set path ./dataset
dupli-hq> set mode duplicates
dupli-hq> run
```

C. File Comparison Mode

```
python commands.py --mode compare --file1 a.png --file2 b.png --auto
```

11. Extending the Tool

To add a new AI model: 1. Add `your_model.py` under `/src/ai_model/` 2. Implement:

```
def load_model(): ...  
def extract_features(file_path, model, transform): ...
```

3. Register it in `loader.module_tree` 4. Optionally, integrate into `scan_duplicates.py` or `commands.py`

12. Expected Behavior Summary

- **Snapshot Mode:** Always generates AIHASH entries + compares with last version
- **Duplicate Mode:** Merges cryptographic + perceptual + embedding similarity
- **Tracker Mode:** Continuously monitors and logs change events
- **Compare Mode:** One-off similarity analysis between two files
- **Logging:** All CLI actions written to `/src/logs/cli_commands.log`